

Lab 3. Specialized Chart Types

PUBH 6199: Visualizing Data with R, Summer 2026

Xindi (Cindy) Hu, ScD

2026-06-04



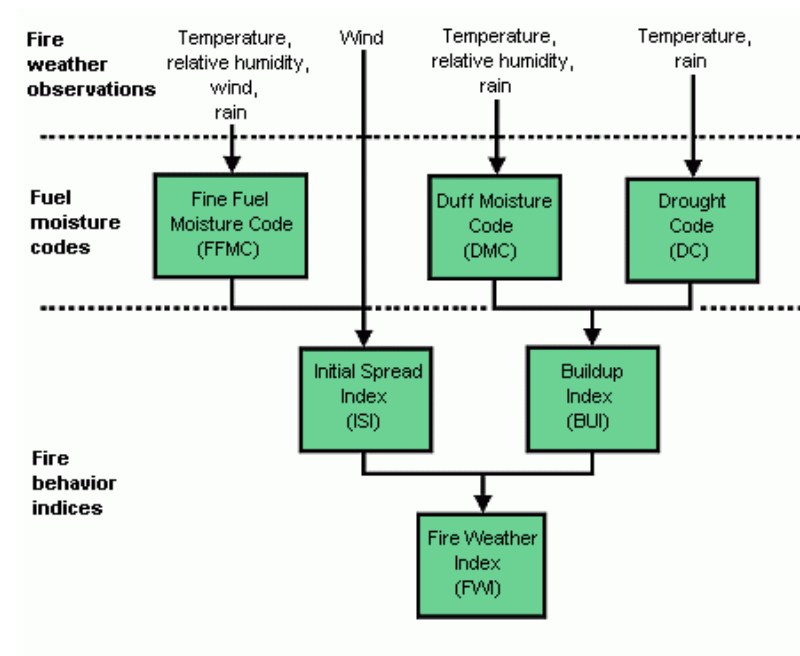
Outline for today

- **Introducing the datasets**
- Review faceted plots
- Introduce radar charts, Sankey diagrams, and parallel coordinates plot
- Dos and Don'ts when designing visualizations



Forest Fires Dataset

- **Source:** UCI Machine Learning Repository
<https://archive.ics.uci.edu/ml/datasets/forest+fires>
- **Context:** Collected from the Montesinho Natural Park in Portugal, this dataset captures weather conditions and fire activity over 200+ days.
- **Variables:**
 - **month, day:** Temporal context
 - **temp, RH, wind, rain:** Daily weather
 - **FFMC, DMC, DC, ISI:** Fire weather indices
 - **area:** Burned area in hectares



Canadian Forest Fire Weather Index (FWI) System

Forest Fires Dataset

```
1 library(tidyverse)
2 forest <- read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/forest-fires/forestfires.csv")
3 glimpse(forest)
```

Rows: 517

Columns: 13

```
$ X      <dbl> 7, 7, 7, 8, 8, 8, 8, 8, 8, 7, 7, 7, 6, 6, 6, 6, 5, 8, 6, 6, 6, 5...
$ Y      <dbl> 5, 4, 4, 6, 6, 6, 6, 6, 6, 5, 5, 5, 5, 5, 5, 5, 5, 5, 4, 4, 4, 4...
$ month  <chr> "mar", "oct", "oct", "mar", "mar", "aug", "aug", "aug", "sep", "...
$ day    <chr> "fri", "tue", "sat", "fri", "sun", "sun", "mon", "mon", "tue", "...
$ FFMC   <dbl> 86.2, 90.6, 90.6, 91.7, 89.3, 92.3, 92.3, 91.5, 91.0, 92.5, 92.5...
$ DMC    <dbl> 26.2, 35.4, 43.7, 33.3, 51.3, 85.3, 88.9, 145.4, 129.5, 88.0, 88...
$ DC     <dbl> 94.3, 669.1, 686.9, 77.5, 102.2, 488.0, 495.6, 608.2, 692.6, 698...
$ ISI    <dbl> 5.1, 6.7, 6.7, 9.0, 9.6, 14.7, 8.5, 10.7, 7.0, 7.1, 7.1, 22.6, 0...
$ temp   <dbl> 8.2, 18.0, 14.6, 8.3, 11.4, 22.2, 24.1, 8.0, 13.1, 22.8, 17.8, 1...
$ RH     <dbl> 51, 33, 33, 97, 99, 29, 27, 86, 63, 40, 51, 38, 72, 42, 21, 44, ...
$ wind   <dbl> 6.7, 0.9, 1.3, 4.0, 1.8, 5.4, 3.1, 2.2, 5.4, 4.0, 7.2, 4.0, 6.7,...
$ rain   <dbl> 0.0, 0.0, 0.0, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,...
$ area   <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
```

Adult Income Dataset

- **Source:** UCI Machine Learning Repository
<https://archive.ics.uci.edu/ml/datasets/adult>
- **Context:** Extracted from 1994 U.S. Census data to predict whether an individual's income exceeds \$50K/year.
- **Variables:**
 - Demographics: `age`, `sex`, `race`, `education`, `marital-status`
 - Employment: `workclass`, `occupation`, `hours-per-week`
 - Target: `income` ($>50K$ or $\leq 50K$)
- **Use in Class:**
 - *Sankey diagram:* Show flows like `education` → `occupation` → `income`
 - *Parallel coordinates:* Visualize patterns across age, hours, and income class



Adult Income Dataset

```
1 adult <- read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data",
2                   col_names = c("age", "workclass", "fnlwgt", "education", "education_num",
3                                 "marital_status", "occupation", "relationship", "race", "sex",
4                                 "capital_gain", "capital_loss", "hours_per_week", "native_country", "income")
5 glimpse(adult)
```

Rows: 32,561

Columns: 15

```
$ age          <dbl> 39, 50, 38, 53, 28, 37, 49, 52, 31, 42, 37, 30, 23, 32,...
$ workclass    <chr> "State-gov", "Self-emp-not-inc", "Private", "Private", ...
$ fnlwgt       <dbl> 77516, 83311, 215646, 234721, 338409, 284582, 160187, 2...
$ education    <chr> "Bachelors", "Bachelors", "HS-grad", "11th", "Bachelors...
$ education_num <dbl> 13, 13, 9, 7, 13, 14, 5, 9, 14, 13, 10, 13, 13, 12, 11,...
$ marital_status <chr> "Never-married", "Married-civ-spouse", "Divorced", "Mar...
$ occupation   <chr> "Adm-clerical", "Exec-managerial", "Handlers-cleaners",...
$ relationship <chr> "Not-in-family", "Husband", "Not-in-family", "Husband",...
$ race         <chr> "White", "White", "White", "Black", "Black", "White", "...
$ sex          <chr> "Male", "Male", "Male", "Male", "Female", "Female", "Fe...
$ capital_gain <dbl> 2174, 0, 0, 0, 0, 0, 0, 0, 14084, 5178, 0, 0, 0, 0, 0, ...
$ capital_loss <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ hours_per_week <dbl> 40, 13, 40, 40, 40, 40, 16, 45, 50, 40, 80, 40, 30, 50,...
$ native_country <chr> "United-States", "United-States", "United-States", "Uni...
$ income       <chr> "<=50K", "<=50K", "<=50K", "<=50K", "<=50K", "<=50K", "..."
```



Outline for today

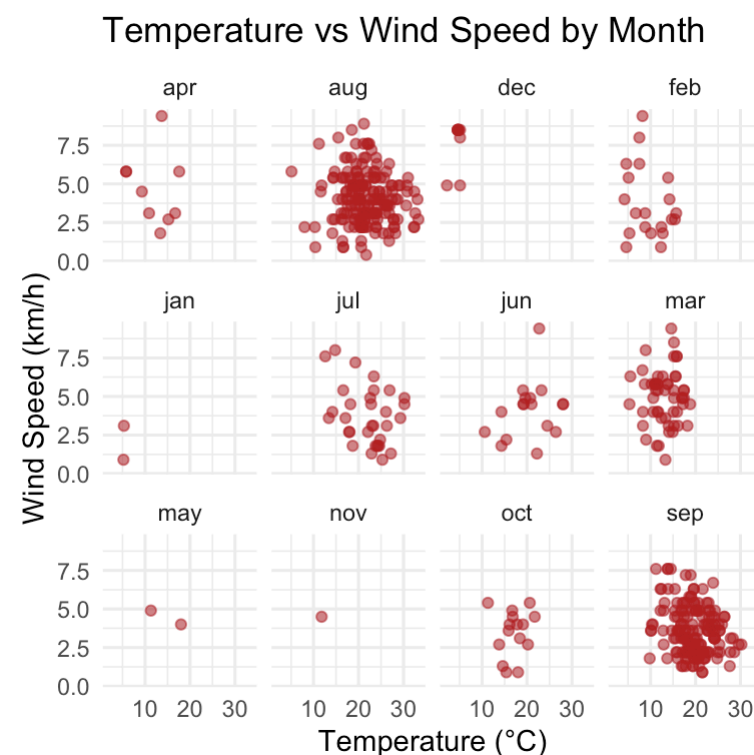
- Introducing the datasets
- **Review faceted plots**
- Introduce radar charts, Sankey diagrams, and parallel coordinates plot
- Dos and Don'ts when designing visualizations



Review: Faceted Plots

In previous lecture/labs, we learned about faceted plots, which allow us to create multiple subplots based on a categorical variable. This is particularly useful for comparing distributions or trends across different groups.

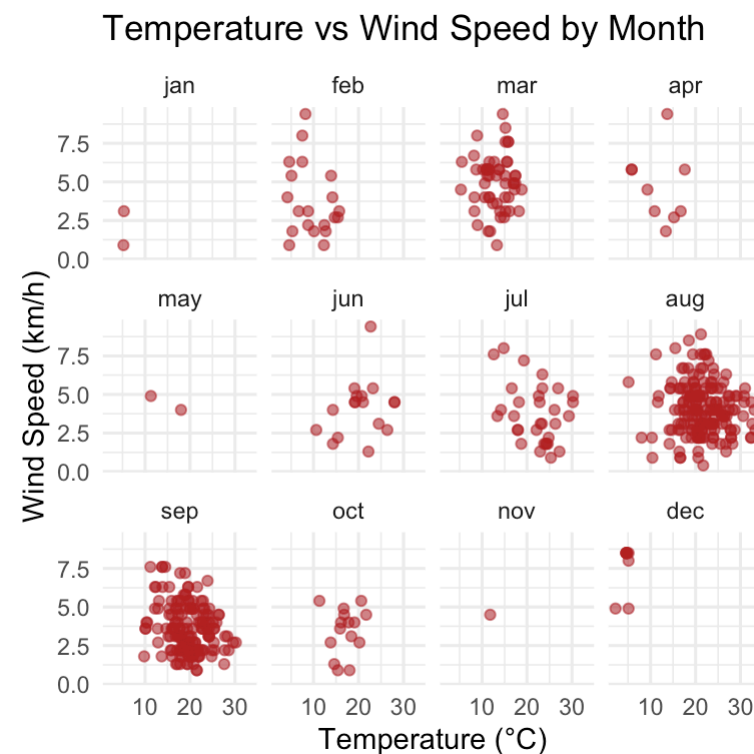
```
1 # Faceted scatter plot: Temp vs Wind, faceted by month
2 ggplot(forest, aes(x = temp, y = wind)) +
3   geom_point(alpha = 0.6, color = "firebrick") +
4   facet_wrap(~ month, ncol = 4) +
5   labs(title = "Temperature vs Wind Speed by Month",
6        x = "Temperature (°C)",
7        y = "Wind Speed (km/h)") +
8   theme_minimal()
```



Review: Faceted Plots

Optional: convert month to ordered factor for nicer facet layout

```
1 forest <- forest %>%
2   mutate(month = factor(month, levels = c("jan", "fe",
3     "jul", "au
4
5 ggplot(forest, aes(x = temp, y = wind)) +
6   geom_point(alpha = 0.6, color = "firebrick") +
7   facet_wrap(~ month, ncol = 4) +
8   labs(title = "Temperature vs Wind Speed by Month",
9     x = "Temperature (°C)",
10    y = "Wind Speed (km/h)") +
11   theme_minimal()
```



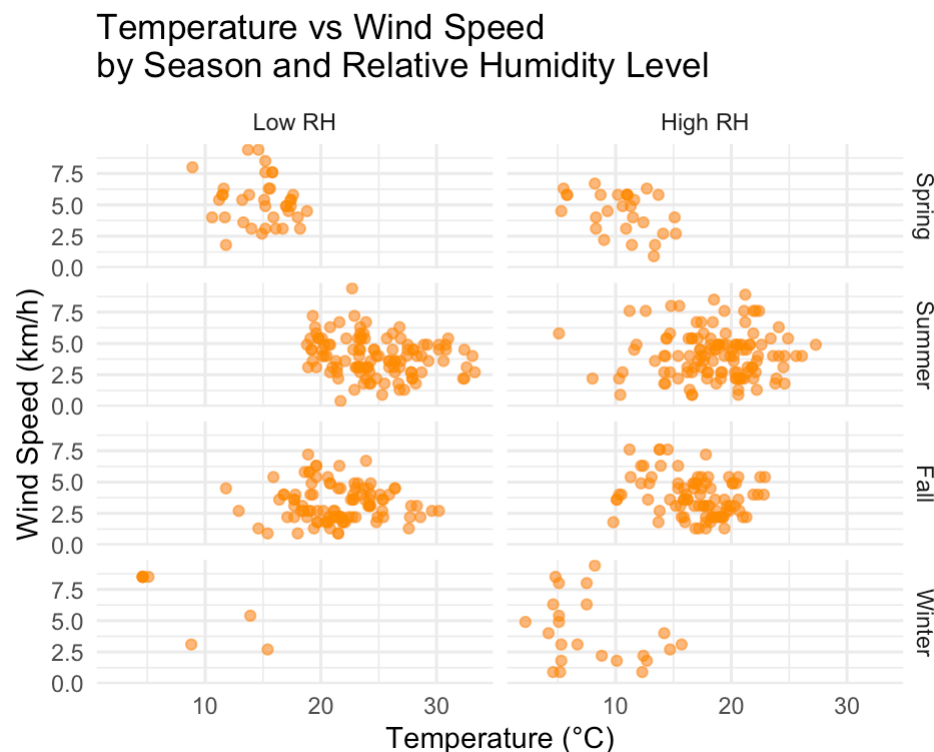
Facet plots by combining two variables

Does dry air (low RH) change the relationship between wind and temperature across seasons? We use `facet_grid()` to create a grid of plots, where each row represents a season and each column represents a relative humidity level.

```

1 forest <- forest %>%
2   mutate(
3     month = tolower(month),
4     season = case_when(
5       month %in% c("dec", "jan", "feb") ~ "Winter",
6       month %in% c("mar", "apr", "may") ~ "Spring",
7       month %in% c("jun", "jul", "aug") ~ "Summer",
8       month %in% c("sep", "oct", "nov") ~ "Fall",
9       TRUE ~ "Unknown"
10    ),
11    RH_level = if_else(RH >= median(RH, na.rm = TRUE), "High RH",
12                      season = factor(season, levels = c("Spring", "Summer", "Fall")
13                      RH_level = factor(RH_level, levels = c("Low RH", "High RH"))
14  )
15
16 # Faceted scatter plot: temp vs wind by season and RH level
17 ggplot(forest, aes(x = temp, y = wind)) +
18   geom_point(alpha = 0.6, color = "darkorange") +
19   facet_grid(rows = vars(season), cols = vars(RH_level)) +
20   labs(
21     title = "Temperature vs Wind Speed\nby Season and Relative Hu
22     x = "Temperature (°C)",
23     y = "Wind Speed (km/h)"
24   ) +
25   theme_minimal(base_size = 11)

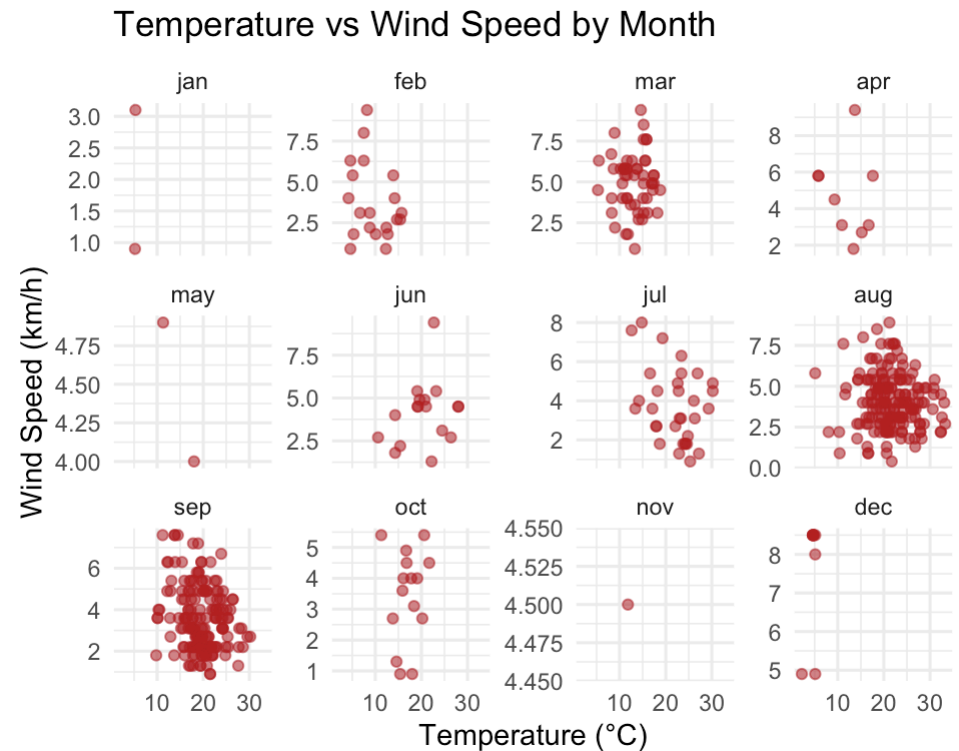
```



Use fixed vs free scales

Using fixed scales can help in comparing values across different facets, while free scales allow each facet to have its own scale, which can be useful when the data varies widely between groups.

```
1 forest <- forest %>%
2   mutate(month = factor(month, levels = c("jan", "fe",
3     "jul", "au
4
5 ggplot(forest, aes(x = temp, y = wind)) +
6   geom_point(alpha = 0.6, color = "firebrick") +
7   facet_wrap(~ month, ncol = 4, scales = "free_y") +
8   labs(title = "Temperature vs Wind Speed by Month",
9     x = "Temperature (°C)",
10    y = "Wind Speed (km/h)") +
11   theme_minimal()
```



 Your turn, get on the bike!
~ *Head over to lab3 notebook!* ~

Outline for today

- Introducing the datasets
- Review faceted plots
- **Introduce radar charts, Sankey diagrams, and parallel coordinates plot**
- Dos and Don'ts when designing visualizations



What is a Radar Chart?

- A radar chart (or spider chart) displays multivariate data on axes starting from the same point.
- Good for comparing profiles across a few categories or groups.
- Useful when variables are on the **same scale** or **normalized**.



Why Use a Radar Chart to explore the forest data?

- Compare **weather profiles** for different fire conditions.
- Visualize **relative intensity** of multiple variables at once (e.g., **temp**, **wind**, **RH**).
- Spot patterns across fire size categories.



Step 1: Load and prepare the data

Forest Fires Dataset This dataset contains 517 observations, each representing a single day of recorded weather conditions and fire activity in the Montesinho Natural Park in Portugal. Each observation includes meteorological variables such as temperature, relative humidity (RH), wind speed, and rainfall, as well as fire weather indices (e.g., FFMC, DMC) and the area burned in hectares. The data spans multiple months and is commonly used to study environmental drivers of wildfire risk.

```

Rows: 517
Columns: 14
$ X          <dbl> 7, 7, 7, 8, 8, 8, 8, 8, 8, 7, 7, 7, 6, 6, 6, 6, 5, 8, 6, 6,...
$ Y          <dbl> 5, 4, 4, 6, 6, 6, 6, 6, 6, 5, 5, 5, 5, 5, 5, 5, 5, 4, 4,...
$ month      <chr> "mar", "oct", "oct", "mar", "mar", "aug", "aug", "aug", "se...
$ day        <chr> "fri", "tue", "sat", "fri", "sun", "sun", "mon", "mon", "tu...
$ FFMC       <dbl> 86.2, 90.6, 90.6, 91.7, 89.3, 92.3, 92.3, 91.5, 91.0, 92.5,...
$ DMC        <dbl> 26.2, 35.4, 43.7, 33.3, 51.3, 85.3, 88.9, 145.4, 129.5, 88....
$ DC         <dbl> 94.3, 669.1, 686.9, 77.5, 102.2, 488.0, 495.6, 608.2, 692.6...
$ ISI        <dbl> 5.1, 6.7, 6.7, 9.0, 9.6, 14.7, 8.5, 10.7, 7.0, 7.1, 7.1, 22...
$ temp       <dbl> 8.2, 18.0, 14.6, 8.3, 11.4, 22.2, 24.1, 8.0, 13.1, 22.8, 17...
$ RH         <dbl> 51, 33, 33, 97, 99, 29, 27, 86, 63, 40, 51, 38, 72, 42, 21,...
$ wind       <dbl> 6.7, 0.9, 1.3, 4.0, 1.8, 5.4, 3.1, 2.2, 5.4, 4.0, 7.2, 4.0,...
$ rain       <dbl> 0.0, 0.0, 0.0, 0.2, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,...
$ area       <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...
$ area_group <chr> "No fire", "No fire", "No fire", "No fire", "No fire", "No

```


Step 2: Summarize and normalize variables

```

1 library(scales)
2 library(fmsb)
3 # Select and normalize variables
4 radar_data <- forest %>%
5   group_by(area_group) %>%
6   summarise(
7     temp = mean(temp),
8     RH = mean(RH),
9     wind = mean(wind),
10    rain = mean(rain)
11  ) %>%
12    # normalize to [0, 1]
13    mutate(across(where(is.numeric), rescale))
14
15 # Convert to fmsb format
16 # fmsb expects first two rows to be max and min
17 radar_df <- radar_data %>%
18   tibble::column_to_rownames("area_group") %>%
19   as.data.frame()
20
21 # fmsb expects first two rows: max then min
22 max_row <- rep(1, ncol(radar_df))
23 min_row <- rep(0, ncol(radar_df))
24 radar_df <- rbind(max = max_row, min = min_row, radar_data)
25

```

	temp	RH	wind	rain
max	1.0000000	1.0000000	1.0000000	1.0000000
min	0.0000000	0.0000000	0.0000000	0.0000000
Large fire	0.2189738	0.0000000	1.0000000	1.0000000
Medium fire	0.2379999	0.6826801	0.2434875	0.1404139
No fire	0.0000000	1.0000000	0.0000000	0.2043269
Small fire	1.0000000	0.4812876	0.2399755	0.0000000

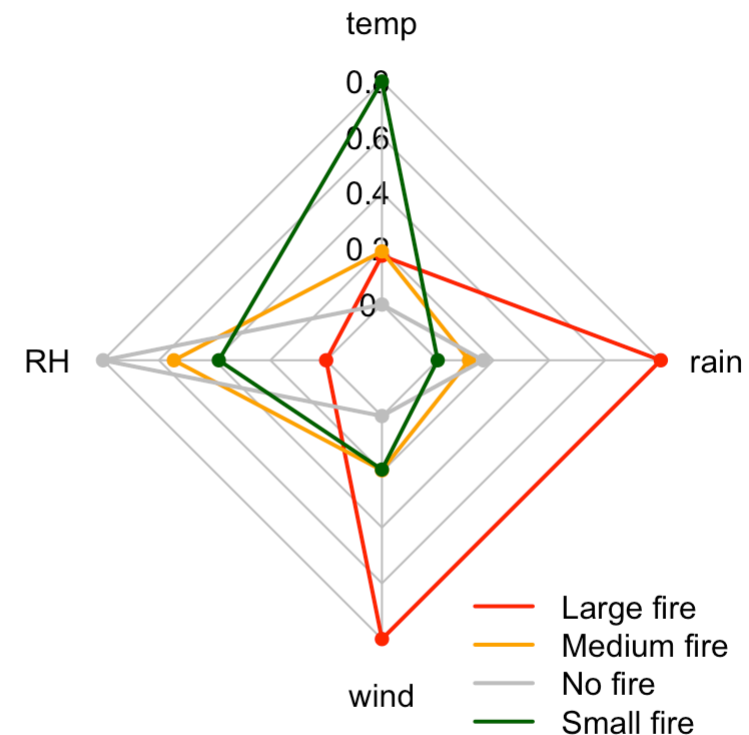
Step 3: Plot the Radar Chart

```

1  radarchart(radar_df,
2             axistype = 1,
3             pcol = c("red", "orange", "grey", "darkgreen"),
4             plwd = 2,
5             plty = 1,
6             cglcol = "grey",
7             cglty = 1,
8             axislabcol = "black",
9             caxislabels = seq(0, 1, 0.2),
10            title = "Weather Profiles by Fire Size")
11
12  legend("bottomright",
13        legend = rownames(radar_df[-c(1,2),]),
14        col = c("red", "orange", "grey", "darkgreen"),
15        lty = 1, lwd = 2, bty = "n")

```

Weather Profiles by Fire Size

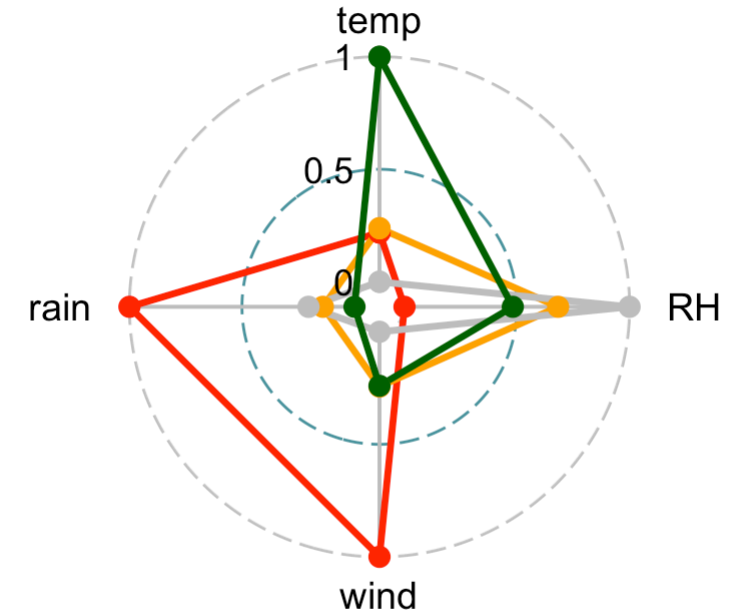


Make radar chart with {ggradar}

Once you have `radar_data` prepared, you can use the `ggradar` package to create a more polished radar chart.

```
1 remotes::install_github("ricardo-bion/ggradar")
2 library(ggradar)
3 ggradar(radar_data,
4         grid.min = 0, grid.mid = 0.5, grid.max = 1,
5         group.line.width = 1.2,
6         group.colours = c('red', 'orange', 'grey',
7         group.point.size = 3,
8         font.radar = "Arial",
9         values.radar = c("0", "0.5", "1"),
10        background.circle.colour = "transparent",
11        legend.position = "bottom") +
12 ggtitle("Weather Profiles by Fire Size") +
13 theme(
14     legend.text = element_text(size = 11),
15     legend.key.width = unit(0.5, "cm")
16 )
```

Weather Profiles by Fire Size



● Large fire ● Medium fire ● No fire ● Small fire

What is a Sankey diagram?

- A **Sankey diagram** visualizes flows between categories.
- The width of each flow is proportional to the count or magnitude.
- Great for showing step-wise processes, transitions, or categorical relationships.



Why Use a Sankey Diagram to explore the **adult** data?

- Visualize **income distribution** across multiple categorical variables.
- Visualize **distributional flows** like:
 - Education → Occupation → Income
 - Gender → Workclass → Income



Step 1: Load and Prepare the Data

Adult Income Dataset This dataset contains 32,561 observations, each representing a single adult individual from the 1994 U.S. Census database. Each observation includes demographic and employment-related variables such as age, sex, education, occupation, and work hours, as well as an income label indicating whether the person earns more or less than \$50,000 per year. The dataset is commonly used to study social and economic patterns, and to explore factors associated with income inequality.

```
1 # Keep only relevant columns
2 adult_sankey <- adult %>%
3   select(sex, education, occupation, income) %>%
4   filter(complete.cases(.))
5
6 flow_data <- adult_sankey %>%
7   count(sex, education, occupation, income) %>%
8   ungroup()
9
10 glimpse(flow_data)
```

Rows: 623

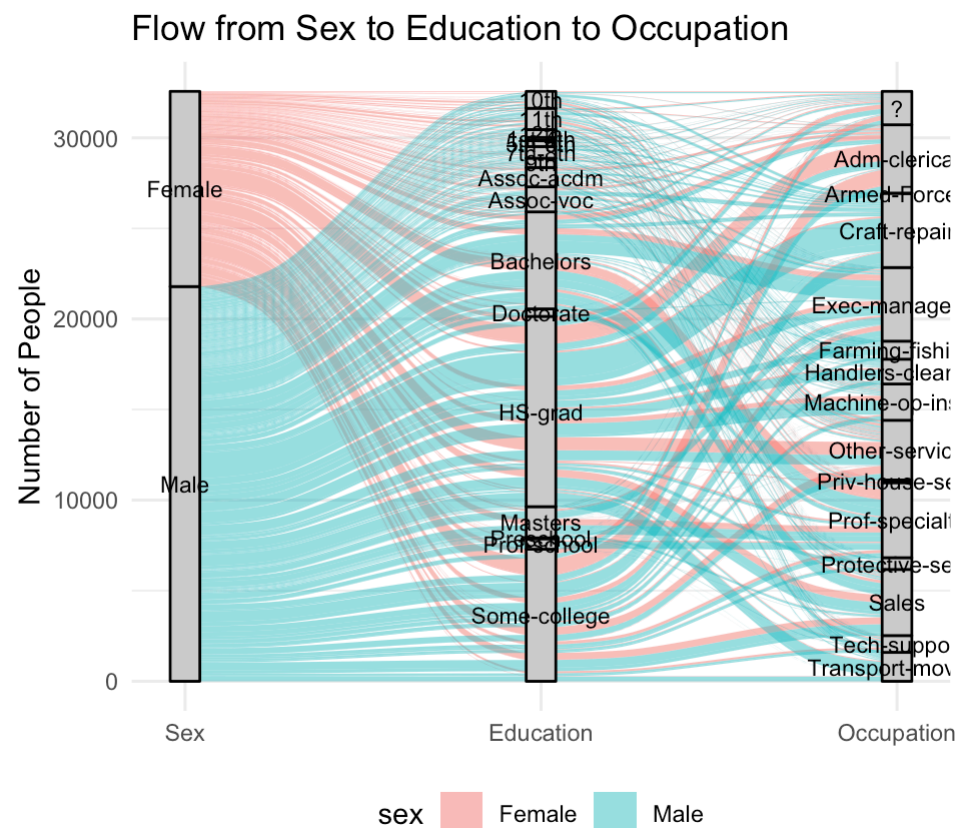
Columns: 5

```
$ sex      <chr> "Female", "Female", "Female", "Female", "Female", "Female", ...
$ education <chr> "10th", "10th", "10th", "10th", "10th", "10th", "10th", "10...
$ occupation <chr> "?", "Adm-clerical", "Craft-repair", "Craft-repair", "Exec-...
```

Step 2: Summarize and plot with `{ggalluvial}`

3-axis Sankey diagrams can quickly become cluttered when categorical variables have too many levels

```
1 library(ggalluvial)
2 library(ggplot2)
3
4 # Basic 3-axis Sankey
5 ggplot(flow_data,
6       aes(axis1 = sex, axis2 = education, axis3 = occupation)) +
7   geom_alluvium(aes(fill = sex), width = 1/12) +
8   geom_stratum(width = 1/12, fill = "grey80", color = "black") +
9   geom_text(stat = "stratum", aes(label = after_stat(n))) +
10  scale_x_discrete(limits = c("Sex", "Education", "Occupation")) +
11  labs(title = "Flow from Sex to Education to Occupation",
12       y = "Number of People") +
13  theme_minimal() +
14  theme(legend.position = "bottom")
```



Step 3: regroup categorical data

Using tools like `forcats::fct_lump()` and binning to reduce the number of flows and make the diagram more readable.

```
1 adult_sankey <- adult_sankey %>%
2   # keep the five most common occupations
3   mutate(occupation = fct_lump(occupation, n = 5)) %>%
4   # group education levels
5   mutate(education_group = case_when(
6     education %in% c("Preschool", "1st-4th", "5th-6th", "7th-8th", "9th", "10th", "11th", "12th") ~ "Less th
7     education == "HS-grad" ~ "HS Graduate",
8     education %in% c("Some-college", "Assoc-acdm", "Assoc-voc") ~ "Some College / Associate",
9     education %in% c("Bachelors") ~ "Bachelor's",
10    education %in% c("Masters", "Doctorate", "Prof-school") ~ "Grad Degree",
11    TRUE ~ "Other"
12  )) %>%
13  mutate(education_group = factor(education_group, levels = c("Less than HS", "HS Graduate", "Some College /
14  flow_data <- adult_sankey %>%
15    count(sex, education_group, occupation, income)
16  glimpse(flow_data)
```

Rows: 120

Columns: 5

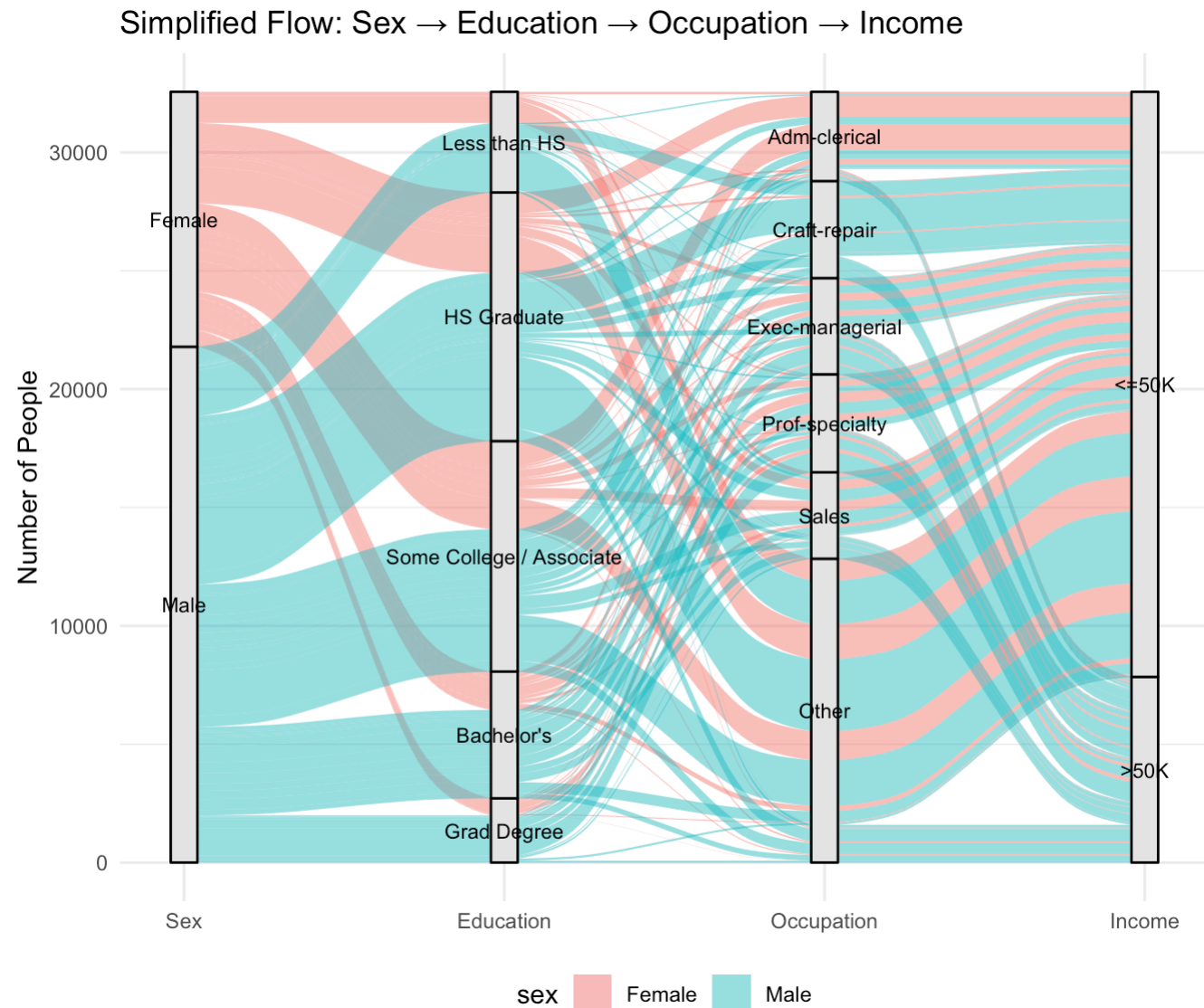
\$ sex	<chr> "Female", "Female", "Female", "Female", "Female", "Fem...
\$ education_group	<fct> Less than HS, Less than HS, Less than HS, Less than HS...
\$ occupation	<fct> Adm-clerical, Adm-clerical, Craft-repair, Craft-repair...
\$ income	<chr> "<=50K", ">50K", "<=50K", ">50K", "<=50K", ">50K", "<=...
\$ n	<int> 108, 1, 29, 5, 25, 3, 20, 1, 191, 1, 925, 12, 883, 85,...

Step 4: replot Sankey diagram

```

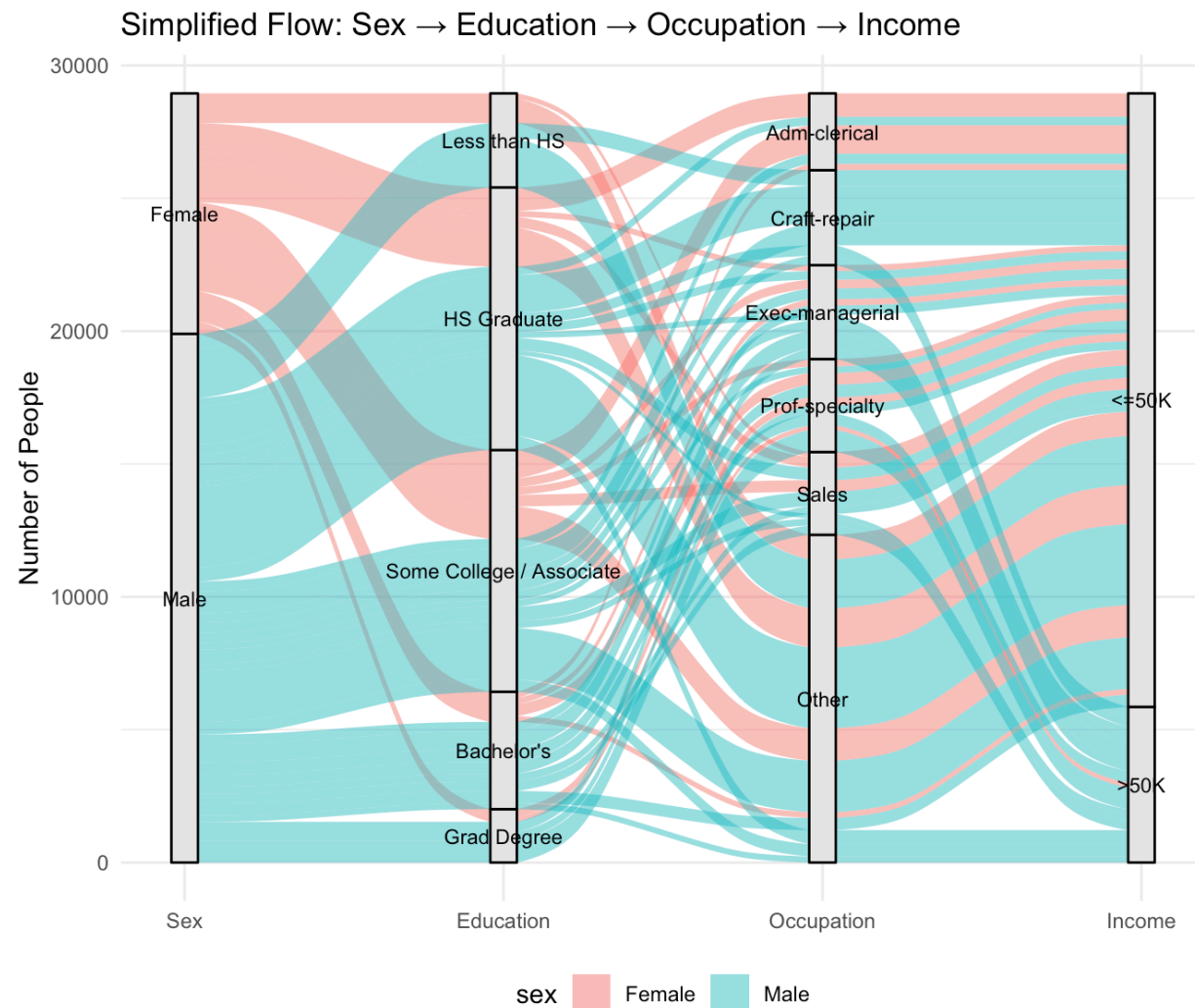
1 # Plot Sankey with 4 axes
2 ggplot(flow_data,
3       aes(axis1 = sex, axis2
4       geom_alluvium(aes(fill =
5       geom_stratum(width = 1/12
6       geom_text(stat = "stratum
7       scale_x_discrete(limits =
8       labs(title = "Simplified
9       y = "Number of People
10      theme_minimal() +
11      theme(legend.position = "

```



Step 5: filter out weak links

```
1 flow_data_filtered <- flow_data %>%
2   filter(n >= 0.005 * sum(n)) # Keep flows >= 0.5% of total
```



What is a Parallel Coordinates Plot (PCP)?

- A visualization for **multivariate data**, especially numeric.
- Each line represents:
 - an **individual** (for raw data), or
 - a **group profile** (for summarized counts).
- Useful for spotting clusters, trends, and differences between groups.



Variation 1: Individual-level PCP

Scaling variable is a crucial step to build a proper parallel coordinates chart. It transforms the raw data to a new scale that is common with other variables, and thus allow to compare them.

```
1 library(GGally)
2 # Prepare numeric individual-level data
3 adult_pc <- adult %>%
4   select(age, education_num, hours_per_week, capital_gain, capital_loss, income) %>%
5   filter(complete.cases(.)) %>%
6   mutate(across(where(is.numeric), rescale)) # normalize for comparability
7
8 # Sample for readability
9 set.seed(123)
10 adult_sample <- adult_pc %>%
11   group_by(income) %>%
12   sample_n(250)
13 glimpse(adult_sample)
```

Rows: 500

Columns: 6

Groups: income [2]

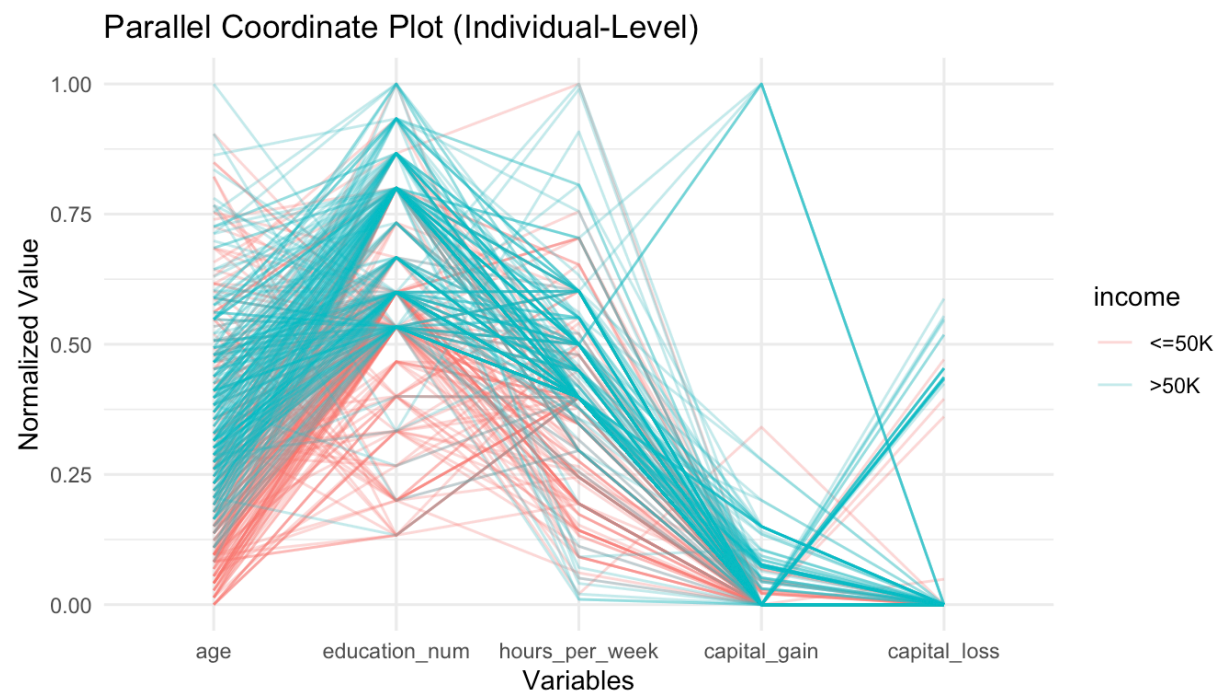
```
$ age          <dbl> 0.61643836, 0.38356164, 0.90410959, 0.24657534, 0.47945...
$ education_num <dbl> 0.53333333, 0.60000000, 0.53333333, 0.66666667, 0.60000000, ...
$ hours_per_week <dbl> 0.3979592, 0.3979592, 0.1938776, 0.2959184, 0.3979592, ...
$ capital_gain  <dbl> 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.000000...
$ capital_loss  <dbl> 0.00000000, 0.00000000, 0.00000000, 0.00000000, 0.00000000, ...
$ income        <chr> "<=50K", "<=50K", "<=50K", "<=50K", "<=50K", "<=50K", "..."
```

Variation 1: Individual-level PCP

```

1 # Individual-level parallel coordinate plot
2 GGally::ggparcoord(
3   data = adult_sample,
4   columns = 1:5,
5   groupColumn = 6,
6   scale = "globalminmax",
7   showPoints = FALSE,
8   alphaLines = 0.3,
9   title = "Parallel Coordinate Plot (Individual-Level)" +
10   theme_minimal() +
11   labs(x = "Variables", y = "Normalized Value")

```



Variation 2: Group-level PCP

In this variation, we summarize the data to create group profiles, which allows us to visualize the average characteristics of each income group across multiple variables.

```
1 # Count and reshape for parallel coordinates
2 edu_occ_income <- adult_sankey %>%
3   count(income, education_group, occupation) %>%
4   pivot_wider(names_from = occupation, values_from = n, values_fill = 0) %>%
5   mutate(income = factor(income, labels = c("<=50K", ">50K")))
6 glimpse(edu_occ_income)
```

Rows: 10

Columns: 8

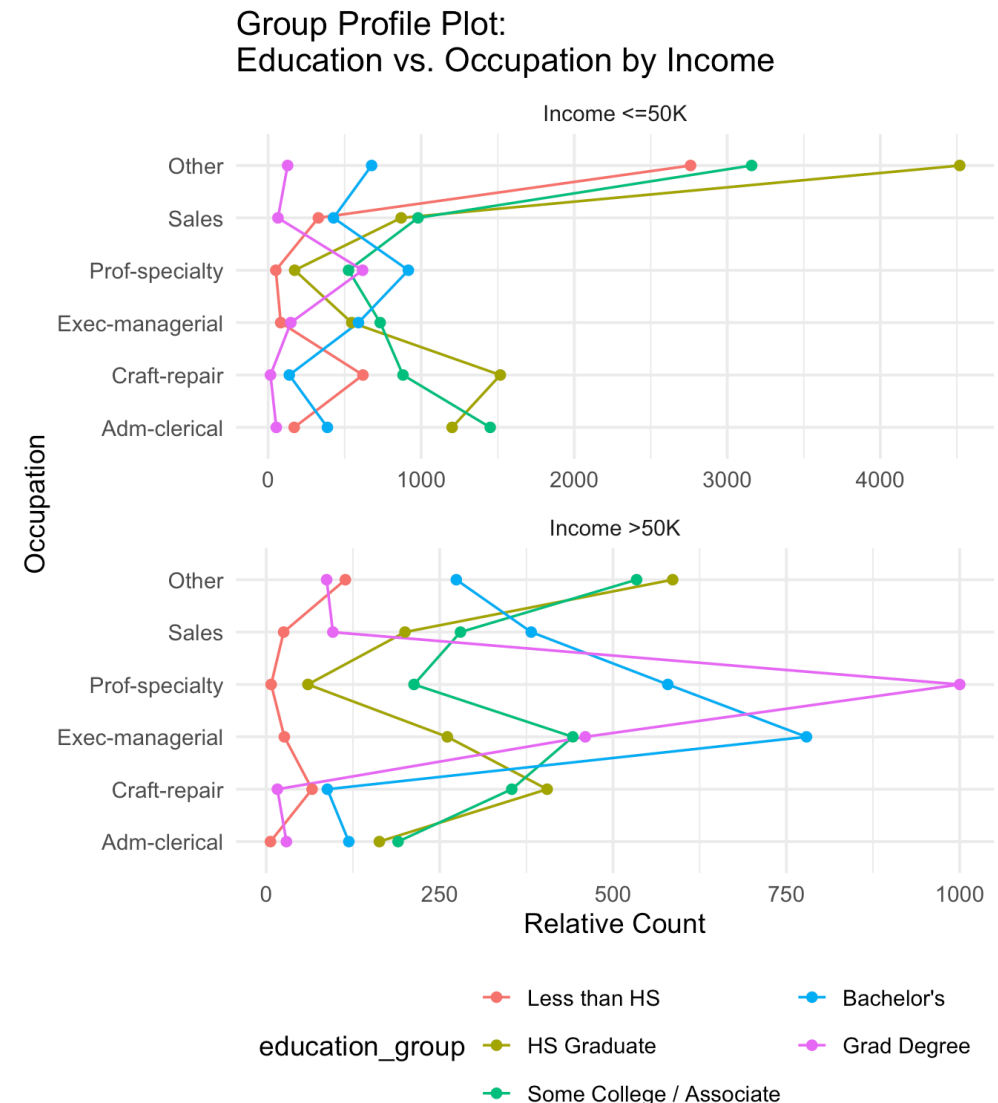
\$ income	<fct> <=50K, <=50K, <=50K, <=50K, <=50K, >50K, >50K, >50K,...
\$ education_group	<fct> Less than HS, HS Graduate, Some College / Associate,...
\$ `Adm-clerical`	<int> 170, 1202, 1451, 387, 53, 6, 163, 190, 119, 29
\$ `Craft-repair`	<int> 619, 1517, 881, 138, 15, 66, 405, 354, 88, 16
\$ `Exec-managerial`	<int> 82, 546, 732, 590, 148, 26, 261, 442, 779, 460
\$ `Prof-specialty`	<int> 50, 173, 525, 916, 617, 7, 60, 213, 579, 1000
\$ Sales	<int> 328, 869, 979, 427, 64, 25, 200, 280, 382, 96
\$ Other	<int> 2760, 4519, 3159, 676, 127, 114, 586, 534, 274, 87

Variation 2: Group-level PCP

```

1 # Group-level profile parallel coordinate plot
2 income_labels <- c(`1` = "Income <=50K", `2` = "Income >50K")
3 GGally::ggparcoord(
4   data = edu_occ_income,
5   columns = 3:ncol(edu_occ_income), # occupation columns
6   groupColumn = 2, #education group
7   scale = "globalminmax",
8   showPoints = TRUE,
9   title = "Group Profile Plot:\nEducation vs. Occupation",
10  theme_minimal(base_size = 14) +
11  facet_wrap(. ~ income, ncol = 1, labeller = as_labeller(income_labels)) +
12  labs(x = "Occupation", y = "Relative Count") +
13  coord_flip() +
14  theme(legend.position = "bottom")+
15  guides(color = guide_legend(nrow = 3))

```



Comparing two variations

Individual-level parallel coordinates plot:

- Each line = one person
- Color by group variable, `income`
- Use this to explore variable interactions and clusters

Group-level parallel coordinates plot:

- Each line = one education group
- Use this to compare how different `education` groups are distributed across `occupations`
- `income` is represented by facets



 Your turn to make some graphs!

~ Head over to lab3 notebook! ~

Outline for today

- Introducing the datasets
- Review faceted plots
- Introduce radar charts, Sankey diagrams, and parallel coordinates plot
- **Dos and Don'ts when designing visualizations**

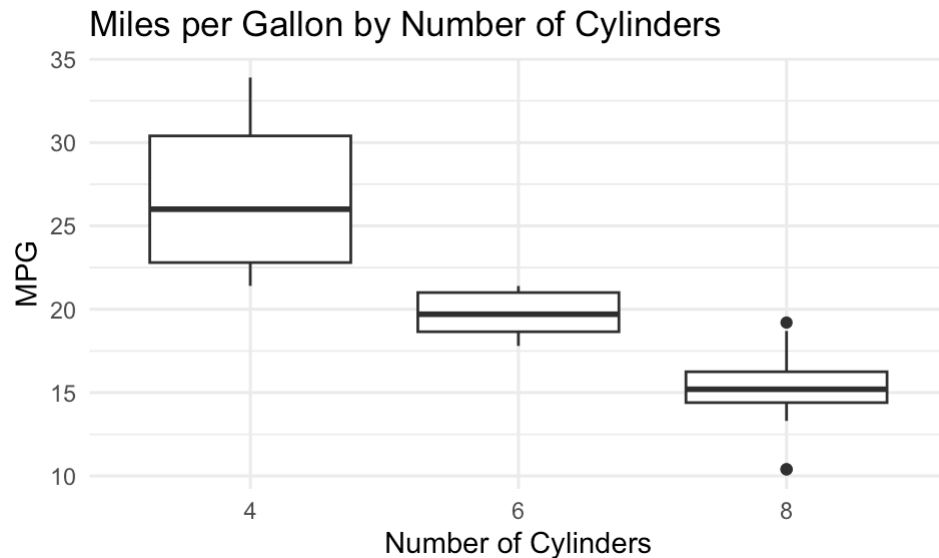


DO: Clear, informative titles and labels



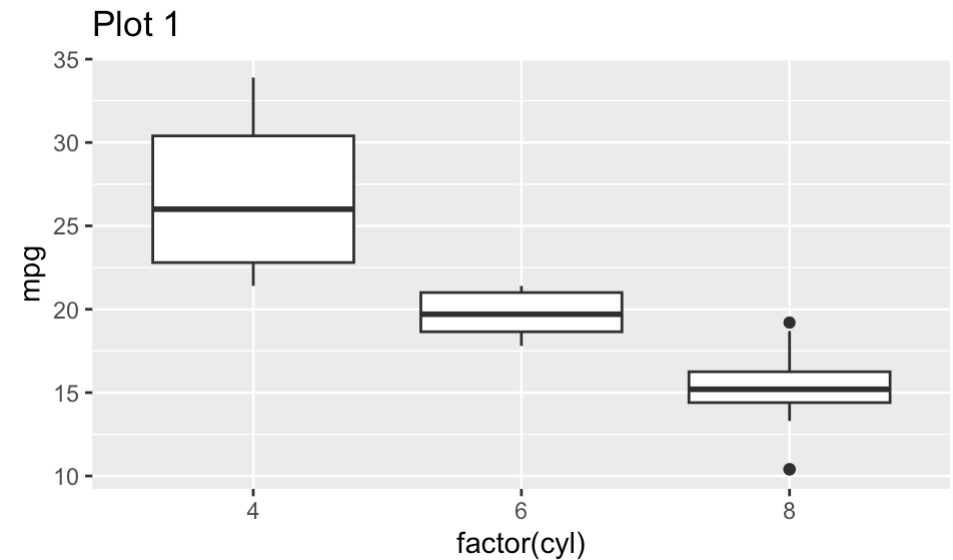
DO

```
1 ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
2   geom_boxplot() +
3   labs(
4     title = "Miles per Gallon by Number of Cylinders",
5     x = "Number of Cylinders",
6     y = "MPG"
7   ) +
8   theme_minimal()
```



Don't

```
1 ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
2   geom_boxplot() +
3   ggtitle("Plot 1") + # Uninformative
4   theme_gray()        # Cluttered default theme
```

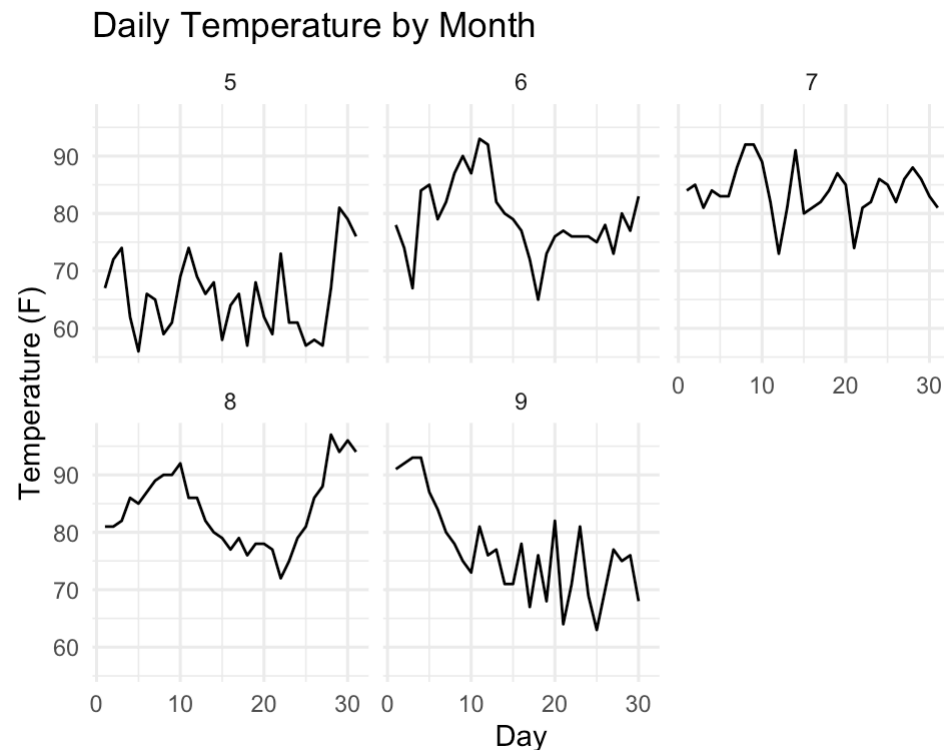


DO: Choose the right chart type



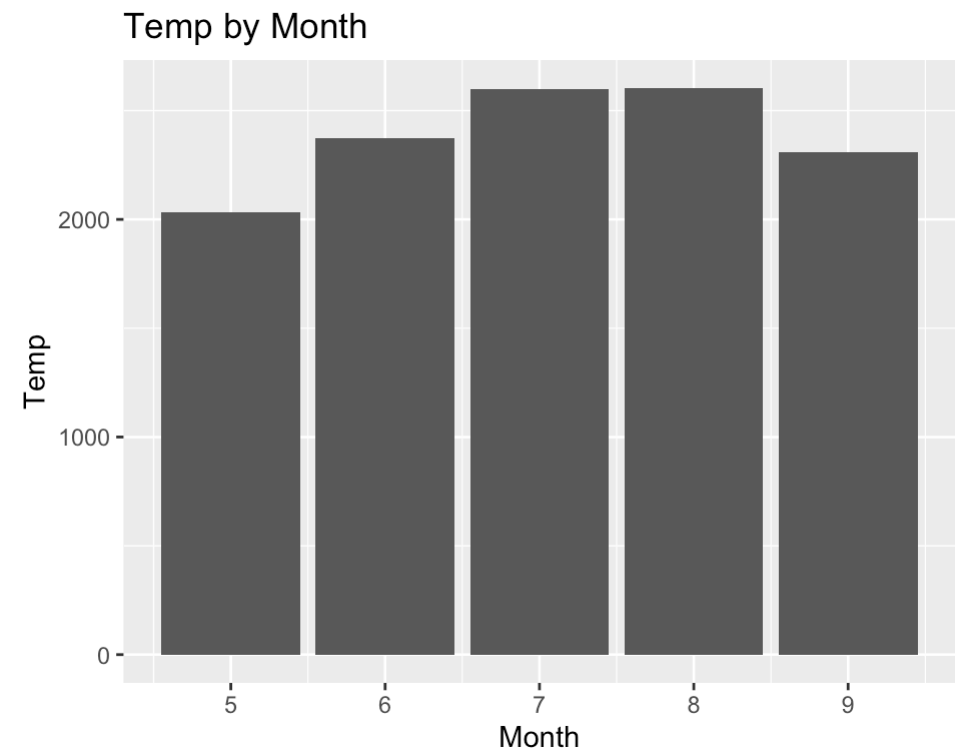
DO

```
1 ggplot(airquality, aes(x = Day, y = Temp)) +
2   geom_line() +
3   facet_wrap(~ Month) +
4   labs(title = "Daily Temperature by Month", x = "Day") +
5   theme_minimal()
```



Don't

```
1 ggplot(airquality, aes(x = Month, y = Temp)) +
2   geom_bar(stat = "identity") +
3   ggtitle("Temp by Month") # Misleading chart type
```



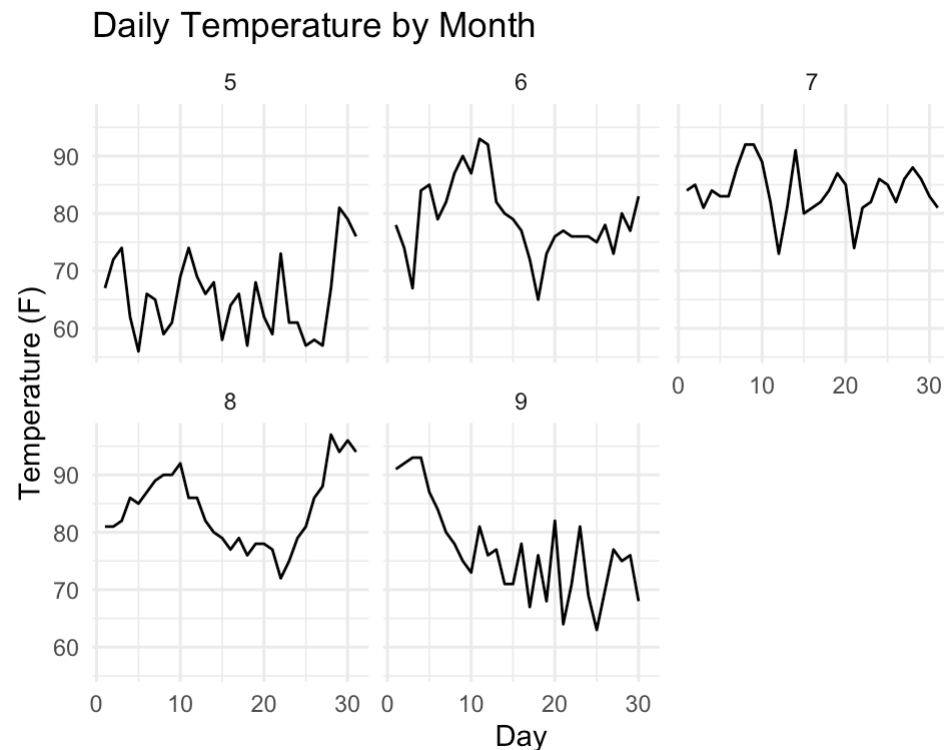
What is this chart showing? Average temperature or total temperature by month?

DO: Choose the right chart type



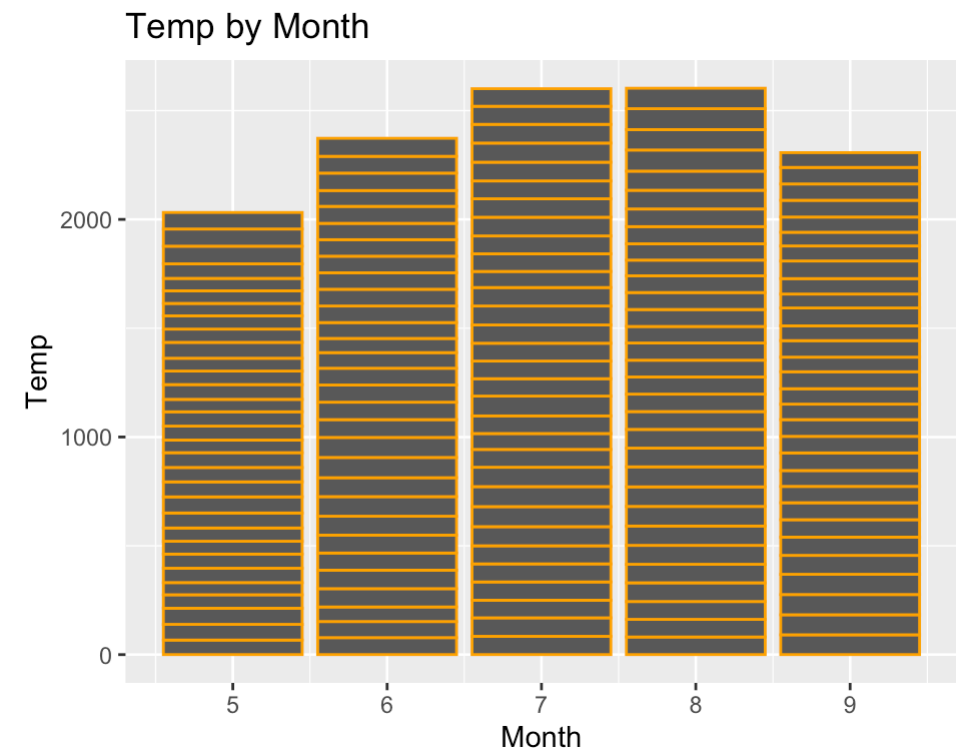
DO

```
1 ggplot(airquality, aes(x = Day, y = Temp)) +
2   geom_line() +
3   facet_wrap(~ Month) +
4   labs(title = "Daily Temperature by Month", x = "Day")
5   theme_minimal()
```



Don't

```
1 ggplot(airquality, aes(x = Month, y = Temp)) +
2   geom_bar(stat = "identity", color = 'orange') +
3   ggtitle("Temp by Month") # Misleading chart type
```

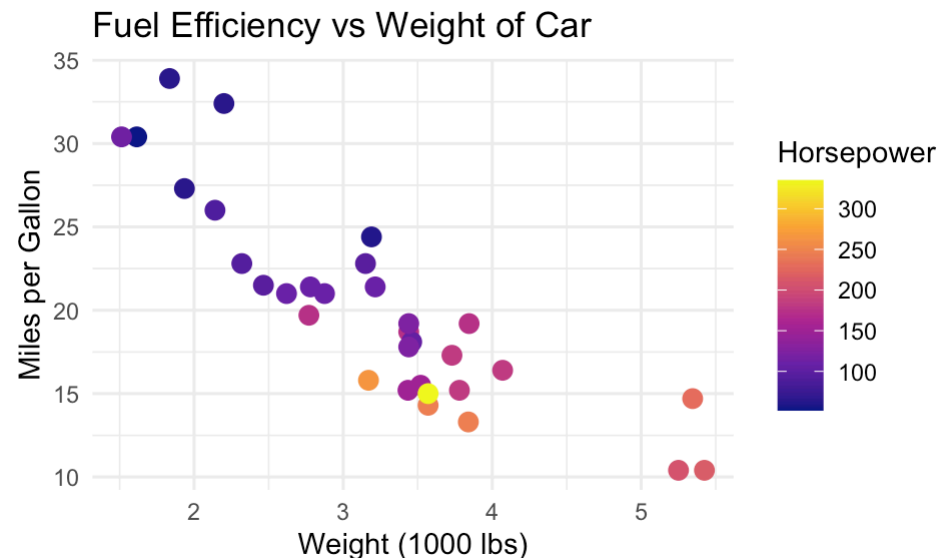


It stacks daily temperature on top of each other, which makes no sense

DO: Use accessible colors

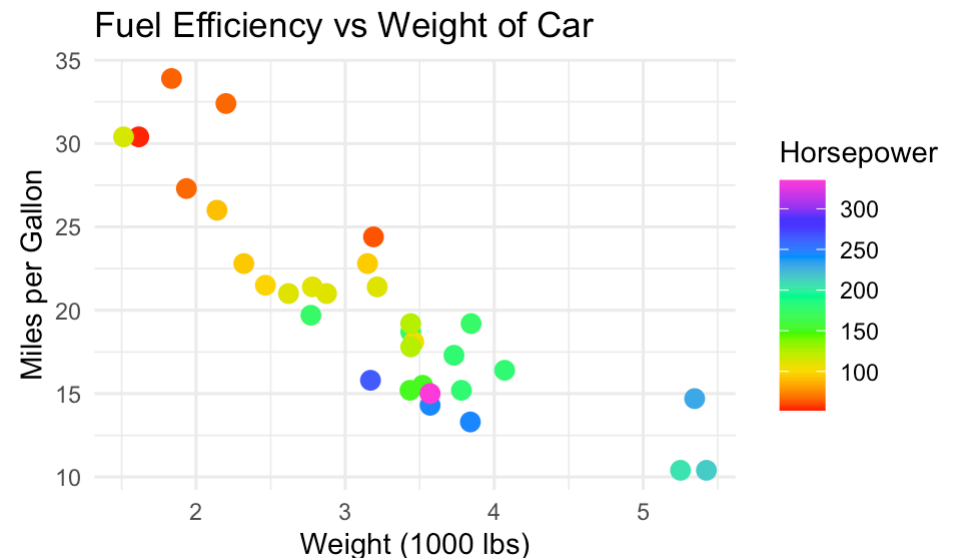
✓ DO use **viridis** scale

```
1 ggplot(mtcars, aes(x = wt, y = mpg, color = hp)) +
2   geom_point(size = 3) +
3   scale_color_viridis_c(option = "plasma") +
4   labs(
5     title = "Fuel Efficiency vs Weight of Car",
6     x = "Weight (1000 lbs)",
7     y = "Miles per Gallon",
8     color = "Horsepower"
9   ) +
10  theme_minimal()
```



✗ Don't use rainbow colors

```
1 ggplot(mtcars, aes(x = wt, y = mpg, color = hp)) +
2   geom_point(size = 3) +
3   scale_color_gradientn(colors = rainbow(7)) +
4   labs(
5     title = "Fuel Efficiency vs Weight of Car",
6     x = "Weight (1000 lbs)",
7     y = "Miles per Gallon",
8     color = "Horsepower"
9   ) +
10  theme_minimal()
```



DO: Order categorical axes intentionally



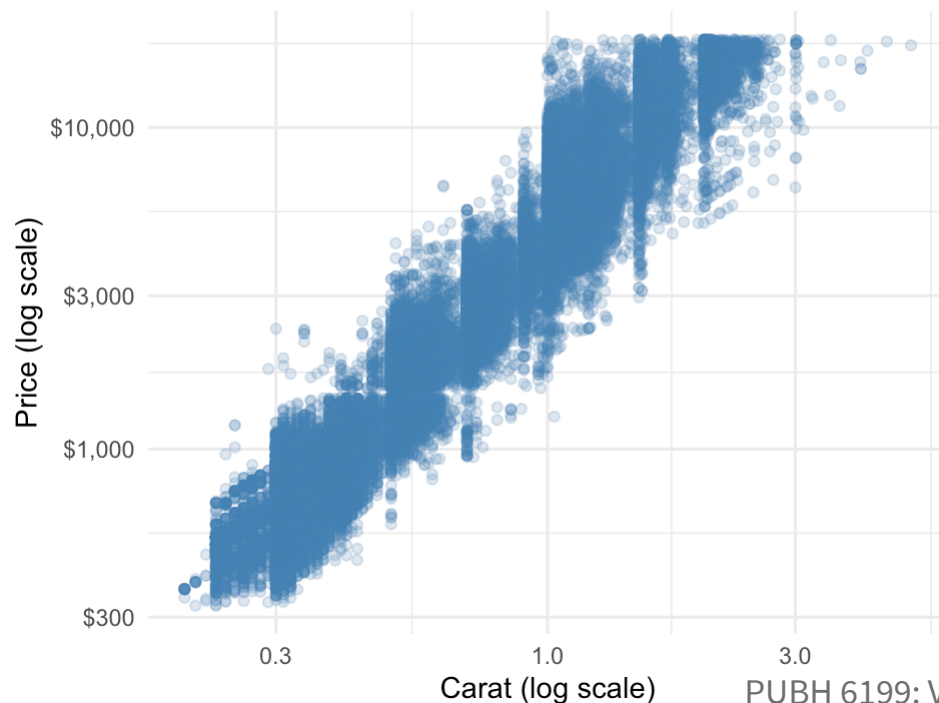
DO: Use appropriate scales

✓ DO log-transform right skewed variables ✗ Don't mindlessly use raw scale

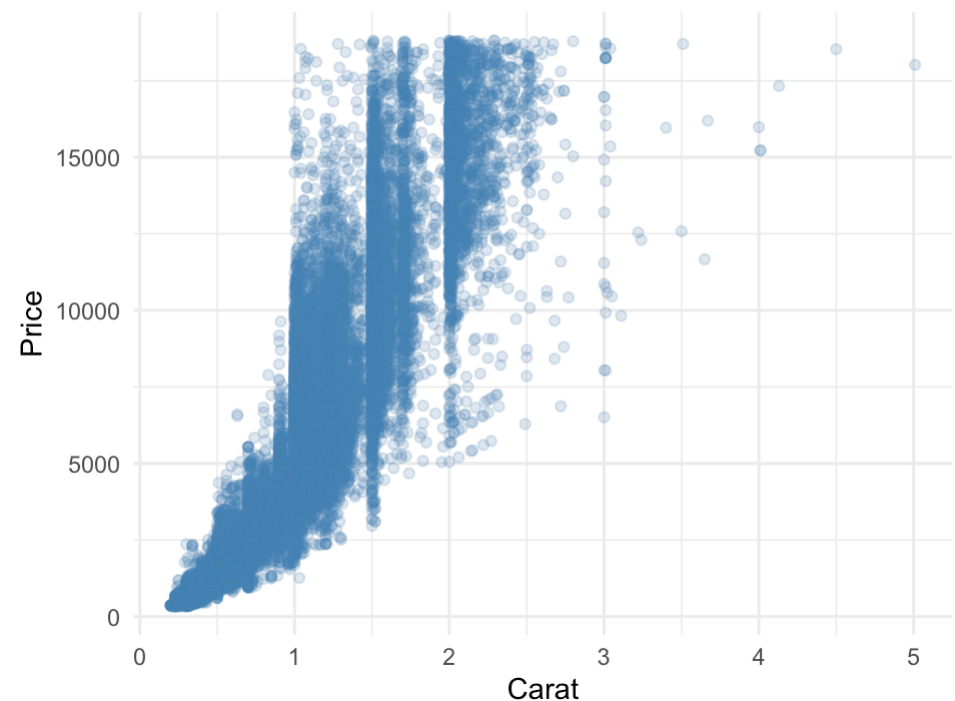
```
1 ggplot(diamonds, aes(x = carat, y = price)) +
2   geom_point(alpha = 0.2, color = "steelblue") +
3   scale_x_log10() +
4   scale_y_log10(labels = scales::dollar_format(accuracy = 1)) +
5   labs(
6     title = "Diamond Price vs Carat (Log-Log Scale)",
7     x = "Carat (log scale)",
8     y = "Price (log scale)") +
9   theme_minimal()
```

```
1 ggplot(diamonds, aes(x = carat, y = price)) +
2   geom_point(alpha = 0.2, color = "steelblue") +
3   labs(
4     title = "Diamond Price vs Carat (Raw Scale)",
5     x = "Carat",
6     y = "Price"
7   ) +
8   theme_minimal()
```

Diamond Price vs Carat (Log-Log Scale)



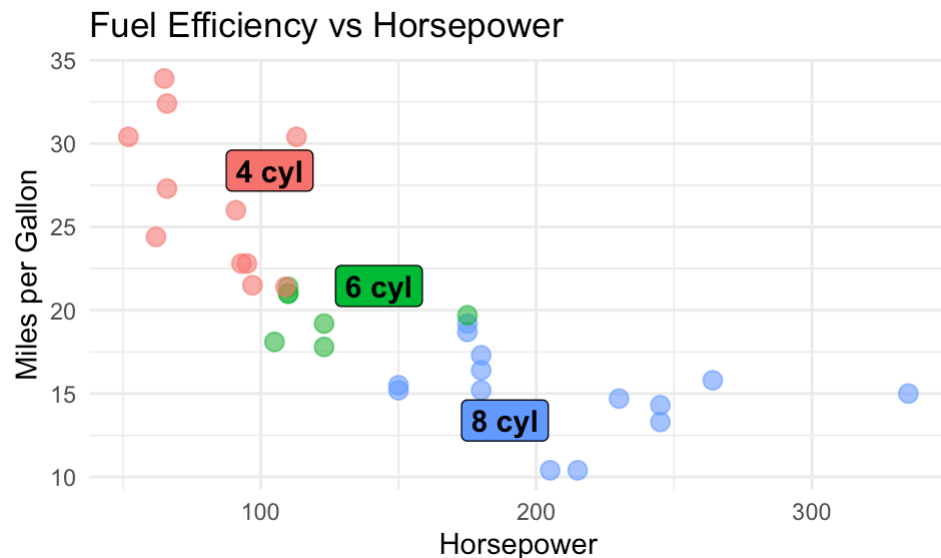
Diamond Price vs Carat (Raw Scale)



DO: Add helpful annotations

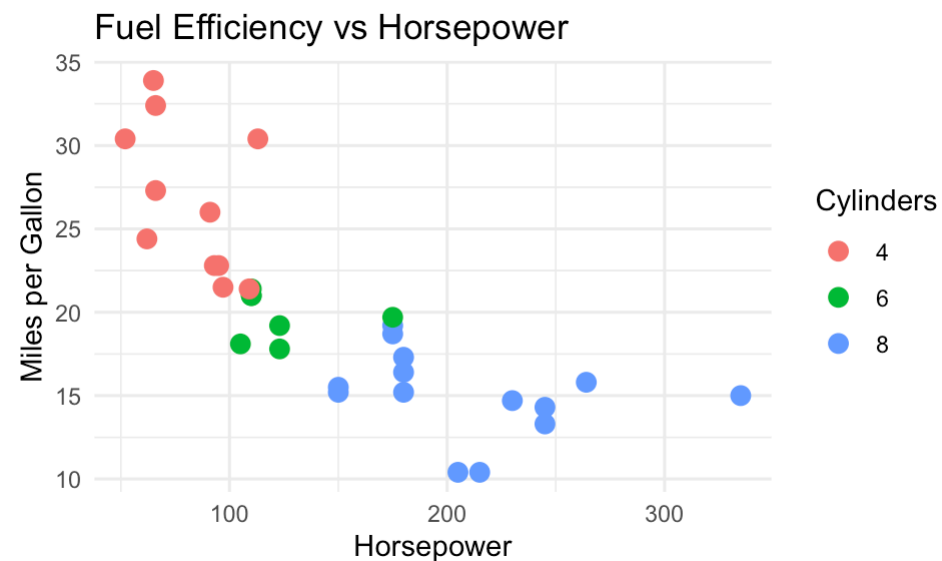
✓ DO use direct labels

```
1 label_df <- mtcars %>%
2   group_by(cyl) %>%
3   summarise(hp = mean(hp), mpg = mean(mpg))
4 ggplot(mtcars, aes(x = hp, y = mpg, color = factor(cyl))) +
5   geom_point(size = 3, alpha = 0.6) +
6   ggrepel::geom_label_repel(
7     data = label_df,
8     aes(label = paste0(cyl, " cyl"), fill = factor(cyl)),
9     show.legend = FALSE, size = 4, fontface = "bold"
10  ) +
11  labs(title = "Fuel Efficiency vs Horsepower", x = "Horsepower", y = "Miles per Gallon") +
12  theme_minimal()
```



✗ Don't only rely on legends

```
1 ggplot(mtcars, aes(x = hp, y = mpg, color = factor(cyl))) +
2   geom_point(size = 3) +
3   labs(
4     title = "Fuel Efficiency vs Horsepower",
5     x = "Horsepower",
6     y = "Miles per Gallon",
7     color = "Cylinders"
8   ) +
9   theme_minimal()
```

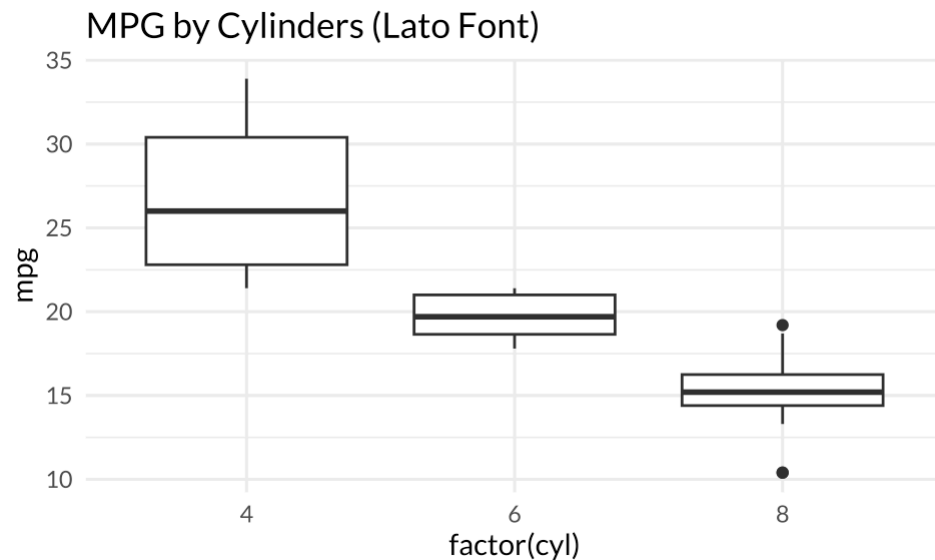


DO: Use clear typeface and fonts



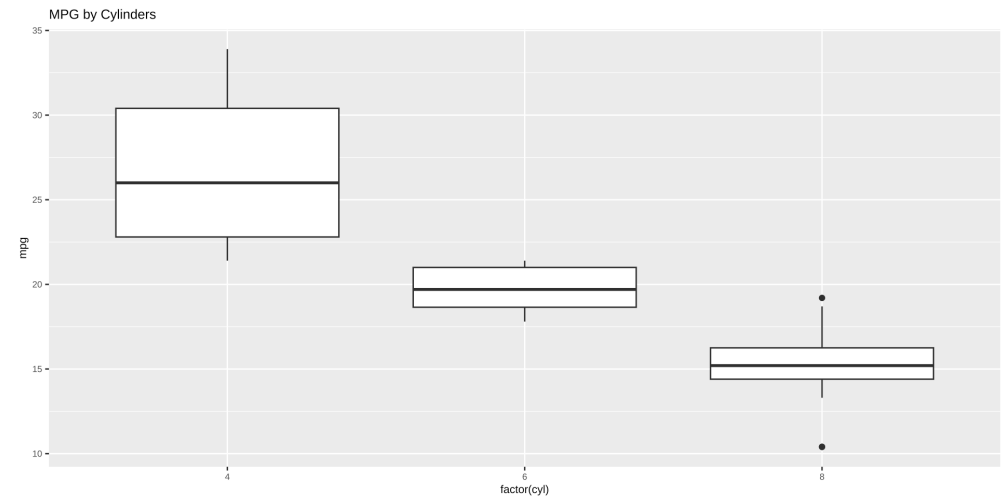
DO

```
1 library(showtext)
2 font_add_google("Lato", "lato")
3 showtext_auto()
4 ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
5   geom_boxplot() +
6   theme_minimal(base_family = "lato") +
7   labs(title = "MPG by Cylinders (Lato Font)")
```



✗ Don't have tiny fonts

```
1 ggplot(mtcars, aes(x = factor(cyl), y = mpg)) +
2   geom_boxplot() +
3   theme(text = element_text(size = 8)) +
4   labs(title = "MPG by Cylinders")
```



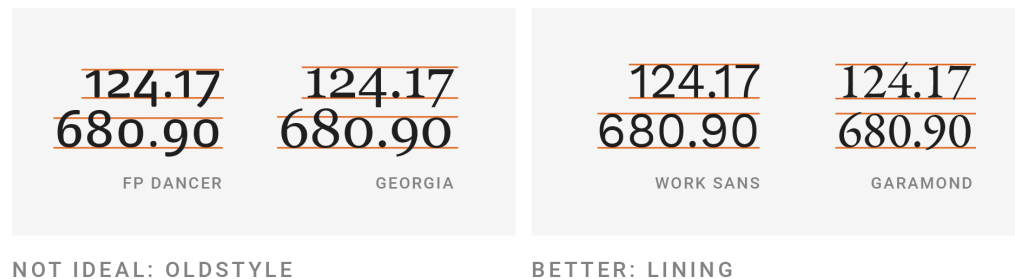
More on typeface



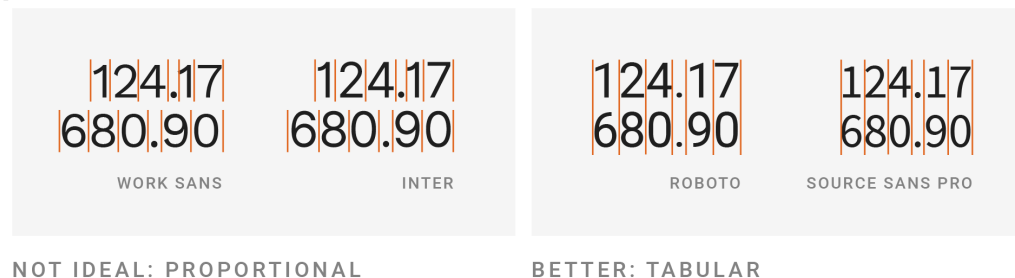
Source: Typography for a better user experience, by [Suvo Ray](#)

Some general rules

- Use **sans-serif** fonts for digital displays (e.g., Arial, Helvetica, Lato).
- **Serif** fonts are typically only used for visualization headlines (e.g., Times New Roman, Georgia).
- Avoid using too many typefaces (just 1-3)
- Use a typeface with lining figures for numerals



- Use a monospaced typeface for numerals



Create hierarchy with font size, weight, and style

YOU

At some point you may come back to read this line or maybe not.

**WILL READ
THIS FIRST.**

And then you will read this line next.

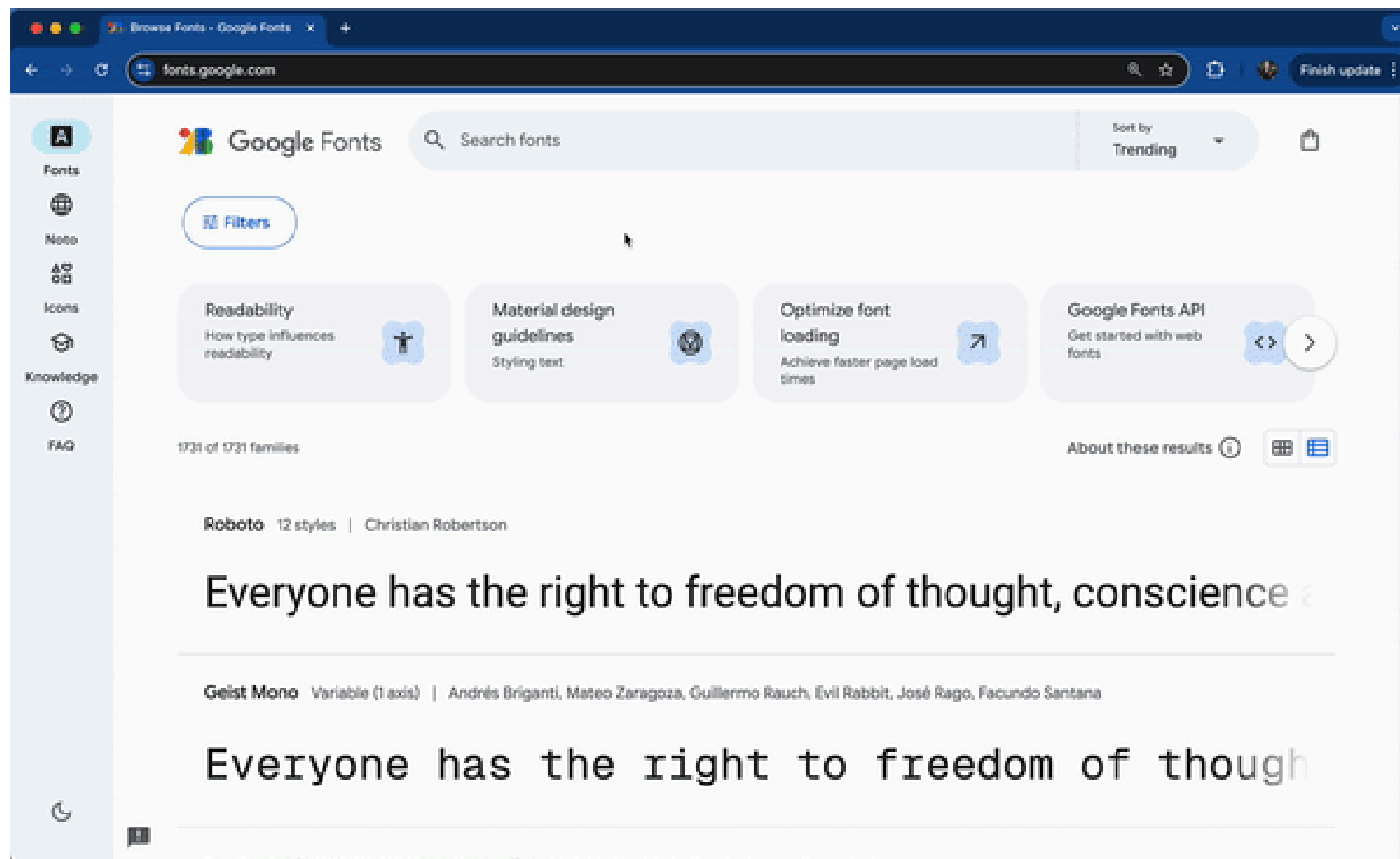
You will go back to read this body copy if you want to know more. It takes the most effort to read because it has a lot of text in a small font in a light weight with tight line spacing. Many people will skip paragraphs like this unless if they aren't engaged right away. This is why it's important to draw attention to your message using visual hierarchy.

**You'll probably
read this before
the paragraph.**

Source: The UX Designer's Guide to Typography by [Chaosamran_Studio](#)



Pick a typeface from Google Fonts



<https://fonts.google.com/>

Source: EDS 240

PUBH 6199: Visualizing Data with R



End-of-Class Survey

 Fill out the end-of-class survey

~ This is the end of Lab 3 ~